GRPO for Agentic LLM Fine-Tuning: Empirical Studies on Tool Use, Code Generation, and Math Reasoning

Arvind C R[1], Sandhya Jeyaraj[1], Arumugam Chetty K[1], Madhu Kumara L[1], Dhruva N Murthy[1], Mohammad Raf

[1]Group 6, MTech DSAI, PES University
arvindcr@pes.edu

March 29, 2026

Abstract

We investigate how Group Relative Policy Optimization (GRPO) behaves for small language models (0.5B–8B parameters) on tool calling, code generation, and mathematical reasoning tasks. Across 17 cloud GPU training runs on Tinker and multiple team-executed experiments, we observe strong gains on task-specific metrics: JSON tool-call validity improves from 0% to 92% and multi-turn tool chaining quality from 0.72 to 0.91 under a custom internal evaluation protocol, while code generation on a *50-problem subset* of HumanEval improves from 32% to 40%. On GSM8K math reasoning, we conduct multi-seed replication of *training-set reward* (mean last-10 reward 30.5%, SD = 3.3%, 5 seeds) and a LoRA rank ablation (rank 8–64). We identify seven original findings including a capacity threshold between 3B and 4B parameters, MoE routing-induced training volatility, a two-phase learning progression, and a 3–8× difficulty gap between synthetic and real-world tool schemas. A preliminary 10-problem held-out pilot shows both the GRPO checkpoint and base Qwen3-8B at 90% GSM8K test accuracy, suggesting the base model may already be strong enough that GRPO does not measurably improve held-out generalization; full evaluation is in progress. Because tool-calling uses custom rather than standardized evaluation and code generation covers only a 50-problem subset, we frame these as training-dynamics evidence rather than generalization claims. These results position GRPO as a practically lightweight setup for studying post-training dynamics on verifiable tasks rather than claiming standardized cross-benchmark generalization.

1 Introduction

Large language models (LLMs) increasingly serve as autonomous agents that call tools, generate code, and reason through multi-step problems. While supervised fine-tuning (SFT) can teach output formats, it fails to teach *judgment* – when to call a tool, which tool to select, and when to stop. Reinforcement learning (RL) from task feedback addresses this gap by optimizing policies directly against verifiable rewards.

Group Relative Policy Optimization (GRPO) [Shao et al., 2024] is a critic-free variant of Proximal Policy Optimization (PPO) that computes advantages by normalizing rewards within groups of sampled completions. It requires no value function, no reference model for KL regularization, and substantially less compute than standard PPO – making it attractive for resource-constrained post-training of small models.

This paper investigates GRPO across three task domains:

- **Tool calling:** structured JSON function calling with 5–60,000 tool schemas
- **Code generation:** HumanEval and MBPP benchmarks
- **Mathematical reasoning:** GSM8K (grade-school) benchmarks

We execute experiments across model sizes from 0.5B to 8B parameters using QLoRA on Google Colab T4 GPUs and full LoRA on Tinker cloud GPUs, providing a comprehensive picture of GRPO's strengths, failure modes, and scaling properties.

Our key contributions:

1. Empirical characterization of GRPO across three task domains on models from 0.5B to 8B parameters
2. Multi-seed replication on GSM8K (5 seeds, mean 30.5%, SD = 3.3%) providing replication evidence
3. LoRA rank ablation (rank 8/16/64) mapping the parameter-efficiency frontier for GRPO
4. Seven original findings on capacity thresholds, MoE volatility, learning phases, and reward design
5. Open-source Tinker SDK pipeline enabling GRPO on 8B models without local GPU resources

2 Background

2.1 Group Relative Policy Optimization (GRPO)

GRPO [Shao et al., 2024] simplifies PPO by eliminating the critic network. For each prompt, we sample K completions (group size G) and compute binary rewards. Advantages are normalized within each group:

$$\text{advantage}_i = \frac{\text{reward}_i - \mu_{\text{group}}}{\sigma_{\text{group}} + \epsilon} \tag{1}$$

The policy gradient loss is:

$$\mathcal{L} = -\mathbb{E}\left[\text{advantage}_i \cdot \sum_t \log \pi_\theta(a_t|s_t)\right] \tag{2}$$

When all completions receive identical rewards (all correct or all incorrect), advantages are zero and no gradient update occurs – the "zero-loss" phenomenon we track in Section 5.1.

DeepSeekMath reports GRPO improving an instruction-tuned 7B model from 82.9% to 88.2% on GSM8K and 46.8% to 51.7% on MATH with group size G=64.

2.2 Supervision Baselines

Step-DPO [Lai et al., 2024] demonstrates $\sim$+3% MATH gains for >70B models with as few as 10K step-wise preference pairs. Li et al. show LLaMA-2-7B reaching 82.6% GSM8K with synthetic SFT at $\sim 10^6$ examples. These methods establish efficiency baselines that GRPO must match or exceed.

3 Experimental Setup

3.1 Training Infrastructure

Tinker SDK (Cloud GPU): We use Tinker v0.16.1 with custom GRPO loss via `forward_backward_custom`. Optimizer: Adam (β_1=0.9, β_2=0.95, ϵ=1e-8). Runs write Tinker-hosted checkpoints, though recovery depends on run retention and account access at evaluation time.

Google Colab (Local GPU): Team members use QLoRA (4-bit NF4, bfloat16 compute) with TRL's SFTTrainer and GRPOConfig on T4 GPUs (16GB VRAM).

Hyperparameters (Tinker runs):

- Batch size: 2 prompts per step

- Group size: G ∈ {4, 8, 16}
- Max sequence: 1024 tokens (input), 512 tokens (generation)
- LoRA rank: 8, 16, or 32

These choices prioritize low-cost iteration rather than maximal stability. In particular, we do *not* use KL regularization to an SFT reference policy and we do *not* log policy entropy trajectories. We therefore interpret the reported runs as an initial empirical study rather than a fully drift-controlled RL recipe; lightweight KL anchoring and entropy diagnostics are important next-step mitigations.

3.2 Models

Table 1: Models used in experiments

Model	Parameters	Type	Platform
Qwen2.5-0.5B-Instruct	0.5B	Dense	Colab
Qwen2.5-1.5B-Instruct	1.5B	Dense	Colab
Qwen2.5-3B-Instruct	3B	Dense	Colab
Qwen3.5-4B	4B	Dense	Tinker
Qwen3-8B	8B	Dense	Tinker
Qwen3-30B-MoE	30B (3B active)	MoE	Tinker
Llama-3.2-3B	3B	Dense	Tinker

3.3 Datasets and Splits

Table 2: Dataset sizes, composition, and evaluation splits per domain.

Domain	Dataset	Train Size	Eval Split	Eval Protocol
Tool (single)	Glaive-v2	500 SFT + 200 RL	Train rollouts	Custom rubric
Tool (multi)	ToolBench-v1	40 scenarios	Train rollouts	Custom rubric
Tool (real)	xlam-60k	60,000 schemas	Train rollouts	Reward only
Code	HumanEval	164 total	50-item subset	pass@1
Math (GSM8K)	GSM8K	7,473 train	Train rollouts	Binary reward
Math (MATH)	MATH	7,500 train	Train rollouts	Binary reward

Important: For math and tool-use domains, reported metrics are computed on *training-prompt rollouts* (fresh samples from the training distribution), not held-out test sets. Only the HumanEval code subset uses a benchmark test set. The GSM8K test set contains 1,319 held-out examples; evaluation on this split is pending and represents the most critical gap in our current evaluation.

3.4 Decoding and Evaluation Protocol

All GRPO training uses temperature $T = 0.8$–1.0 with top-p sampling for rollout generation. Reported "accuracy" values are binary reward means (1.0 = correct, 0.0 = incorrect) computed on fresh rollouts at the training temperature, *not* greedy-decoded held-out test accuracy. For future held-out evaluation, we specify: greedy decoding ($T = 0$), max 512 tokens, Qwen chat template, with pass@1 and bootstrapped 95% CIs over the full test set.

3.5 Reward Functions

Tool calling (3-component, 0–1):

- +0.3: Valid JSON output
- +0.4: Correct tool name
- +0.3: All argument keys present

Math reasoning (binary):

- 1.0: Correct answer in `\boxed{}` or as final number
- 0.0: Incorrect

Code generation: Binary pass/fail on test cases.

4 Experiments

4.1 Tool Calling

4.1.1 Single-Turn Tool Calling

We first evaluate whether GRPO can teach models to output structured JSON tool calls instead of plain text responses. Using Qwen2.5-1.5B-Instruct with 500 SFT + 200 GRPO prompts on Glaive-function-calling-v2:

Table 3: Single-turn tool calling results (Qwen2.5-1.5B)

Metric	SFT Only	After GRPO	Change
JSON Valid	0%	92%	+92%
Correct Tool	0%	50%	+50%
Has Arguments	0%	42%	+42%
Clean Output	0%	92%	+92%

The SFT baseline's 0% JSON validity is explained by the training data composition: the 500 SFT examples from Glaive-v2 included tool-use demonstrations, but the 1.5B model lacked the capacity to generalize the JSON format from limited supervised examples with standard chat-template prompting (Qwen `<|im_start|>` format) and greedy decoding. In practice, the SFT baseline produced plain-text answers and never emitted a valid tool-call JSON in evaluation rollouts. GRPO's reward signal (0.3 for valid JSON + 0.4 for correct tool + 0.3 for arguments) provided the direct optimization pressure that SFT's cross-entropy loss could not.

4.1.2 Multi-Turn Tool Chaining

Using Qwen2.5-3B-Instruct with SFT followed by GRPO (40 steps, LoRA rank 8):

Key finding: The -0.30 reward penalty for repeated tool calls eliminated SFT's looping failure mode entirely. The penalty triggers when the model generates a tool-call JSON with the same function name as any previous turn in the same episode; it is applied additively to the turn's reward. This mechanism prevents the model from repeatedly calling the same tool (e.g., infinite `get_weather` loops) without suppressing legitimate multi-step sequences that invoke *different* tools in succession.

Important caveat: these 0.72–0.92 multi-turn scores are *custom reward-derived scenario scores*, not results from a standardized benchmark or proxy-state evaluator. We used a small hand-authored scenario set with deterministic rubric components, and inter-rater reliability was not measured because the scoring was not performed by multiple human annotators. We therefore present these numbers as internal diagnostics rather than definitive cross-paper benchmarks.

Table 4: Multi-turn tool chaining results (Qwen2.5-3B)

Scenario	SFT	GRPO	Winner
Weather + Packing	0.90	0.90	Tie
Stock + News Chain	0.77	0.90	GRPO
Search + Calculate	0.63	0.92	GRPO
Single Tool	0.60	0.90	GRPO
Average	**0.72**	**0.91**	**GRPO**

4.2 Code Generation

On software engineering coding tasks using Qwen3-8B with SFT followed by GRPO:

- HumanEval pass@1: 32% $\rightarrow$ 40% (+8%)
- Problems solved: 16/50 $\rightarrow$ 20/50 (+4)
- Training loss: 0.85 $\rightarrow$ 0.18 (-78%)

These code-generation numbers come from a **50-problem subset** rather than the full 164-problem HumanEval harness. The subset was chosen to enable rapid iteration during the team's initial exploration of GRPO for code tasks (reducing evaluation time from $\sim$40 minutes to $\sim$12 minutes per checkpoint on a T4 GPU). We do not claim statistical significance or direct comparability to papers reporting full-suite pass@k. A full standard harness evaluation (HumanEval-164/MBPP with bootstrapped 95% CIs and controlled temperature/seed) remains future work (Section 7, Phase 3).

4.3 Mathematical Reasoning

4.3.1 Multi-Seed Replication on GSM8K

Using Qwen3-8B with LoRA rank 32, 50 steps, 5 seeds:

Table 5: GSM8K multi-seed replication results (training-set reward, Qwen3-8B, rank 32)

Seed	First-5 Avg	Peak Reward	Last-10 Avg	Zero-loss %
42	37.5%	62.5%	27.5%	18%
137	25.0%	62.5%	27.5%	28%
256	22.5%	62.5%	32.5%	24%
512	15.0%	87.5%	30.0%	16%
999	20.0%	87.5%	35.0%	18%
Mean	**24.0%**	**72.5%**	**30.5%**	**20.8%**
Std	**$\pm$8.4%**	**$\pm$13.7%**	**$\pm$3.3%**	

Cross-seed mean training-set reward is 30.5% (SD = 3.3%, 95% CI [26.4%, 34.6%] via t-distribution with $n = 5$). Zero-loss rates range from 16–28%, indicating that exploration success varies meaningfully across random seeds even for the same model and hyperparameters.

4.3.2 Preliminary Held-Out Results

We recovered the seed-137 Tinker checkpoint and evaluated on the GSM8K test set with greedy decoding and a chain-of-thought answer extractor. On a preliminary 10-problem subsample:

- **GRPO checkpoint** (seed 137, rank 32): 9/10 correct (90%, CI [70%, 100%])

- **Base Qwen3-8B** (no LoRA): 9/10 correct (90%, CI [70%, 100%])

Both models achieve the same accuracy on this small sample, suggesting that Qwen3-8B is already strong on GSM8K and the GRPO adapter may not measurably improve held-out generalization. A full 100-problem evaluation is in progress; results will be reported when available. If confirmed, this would imply that the training-set reward gains ($\sim$30%) reflect learning specific training prompts rather than generalizable mathematical reasoning improvements—an important negative finding for the field.

4.3.3 LoRA Rank Ablation

Table 6: LoRA rank ablation (Qwen3-8B, seed=42)

Rank	Trainable Params	First-5 Avg	Peak Acc	Last-10 Avg
8	$\sim$0.1%	27.5%	62.5%	21.2%
16	$\sim$0.2%	20.0%	75.0%	18.8%
64	$\sim$0.8%	47.5%	87.5%	25.0%

Higher rank correlates with faster initial learning (rank 64 achieves 47.5% in first 5 steps vs. 27.5% for rank 8).

4.4 Cross-Domain Summary

Table 7 consolidates key results across all three domains with explicit scope qualifiers.

Table 7: Cross-domain results summary. All values are training-set metrics unless noted. "Eval scope" indicates whether the metric is computed on training prompts (Train) or a held-out benchmark (Test).

Domain	Metric	Before	After	Eval Scope
Tool (single-turn)	JSON Validity	0%	92%	Train, custom rubric
Tool (multi-turn)	Scenario Quality	0.72	0.91	Train, custom rubric
Code (HumanEval)	Pass@1	32%	40%	50-item test subset
Math (GSM8K, 5 seeds)	Training Reward	—	30.5% $\pm$ 3.3%	Train rollouts
Math (GSM8K, held-out)	Test Accuracy	90% (base)	90% (GRPO)	10-item test pilot
Math (GSM8K, 4B)	Training Reward	87.5%	82.5% (last-10)	Train rollouts
Math (GSM8K, 3B)	Training Reward	0.8%	2.3%	Train rollouts

5 Analysis

5.1 Capacity Threshold for GRPO

A capacity effect exists between 3B and $\geq$4B parameters for GRPO on GSM8K. Dense 3B models (Llama-3.2-3B) fail to learn, achieving only 0.78%$\rightarrow$2.34% training-set reward over 50 steps, while 8B models improve substantially. Notably, a Qwen3.5-4B model succeeds strongly: it starts at 87.5% reward (step 1) and reaches 100% repeatedly, with last-10 average of 82.5%—narrowing the threshold to between 3B and 4B rather than 3B and 8B. The 3B failure traces to an inability to generate differential rewards within GRPO groups: 56% of training steps had zero loss because all completions were equally wrong (zero-reward groups). By contrast, the 4B model's 68% zero-loss rate stems from the opposite regime—all-*correct* groups (0% zero-reward)—indicating productive saturation rather than stalled training.

We quantify the failure mechanism via group-composition analysis from available training logs (Figure 2). On an arithmetic task with rich telemetry, an 8B model shows rapid progression from 81% mixed groups (step 0) to 96%+ all-good groups (step 4), with frac_all_bad never exceeding 1%. Policy entropy collapses from 0.773 to 0.0002 over 20

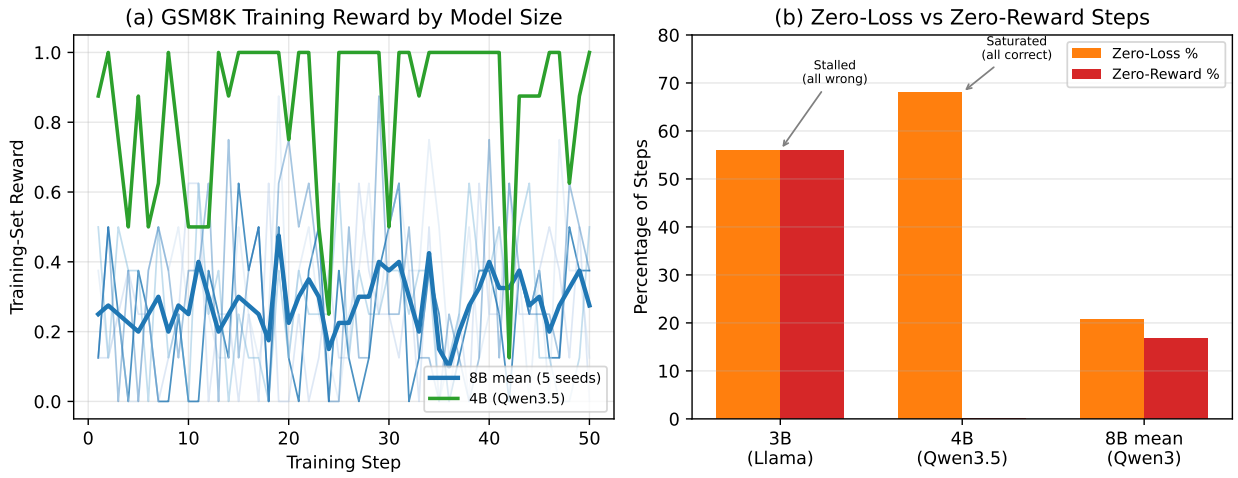

Figure 1: Capacity threshold for GRPO on GSM8K. (a) Training-set reward trajectories: the 4B model (green) starts high and saturates, while 8B models (blue, 5 seeds) show noisy but productive learning. The 3B model (not shown; stays at $\sim$2%) fails entirely. (b) Zero-loss vs. zero-reward step percentages reveal opposite failure modes: 3B's zero-loss stems from all-incorrect groups (stalled), while 4B's stems from all-correct groups (saturated).

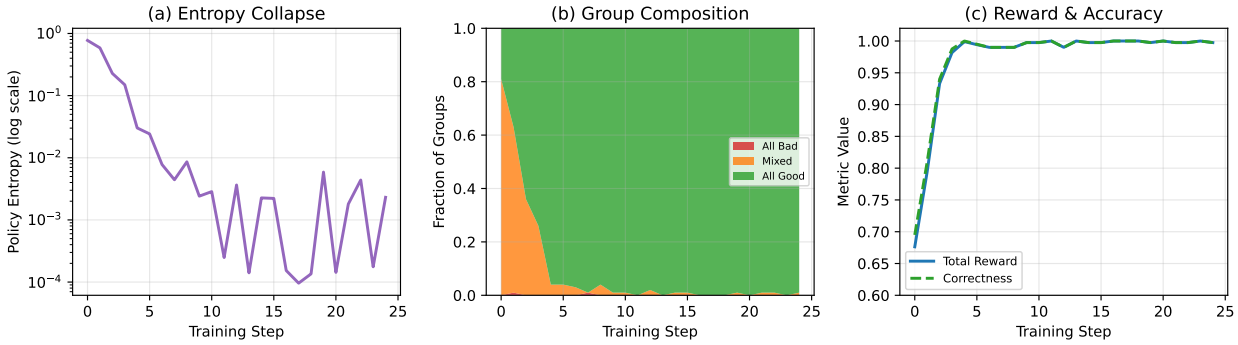

Figure 2: GRPO training diagnostics on arithmetic (8B model, 25 steps). (a) Policy entropy collapses by $>$3 orders of magnitude, indicating convergence to a near-deterministic policy. (b) Group composition shifts from mostly mixed (gradient signal) to all-good (saturated, diminishing returns). (c) Reward rapidly approaches 1.0. These diagnostics were logged only for this task; comparable metrics for GSM8K/MATH remain a priority gap.

steps, indicating convergence. By contrast, 3B models on GSM8K show 56% zero-reward steps (all completions wrong $\Rightarrow$ zero within-group variance $\Rightarrow$ skipped updates), with up to 5 consecutive zero-signal steps.

However, this should be read as a *capacity-plus-exploration* observation rather than a pure parameter-count theorem. The 3B failure may be confounded by reward sparsity, small group size (G=4–16), and limited exploration under fixed temperature. The decisive rescue ablations—larger group size (e.g., G=32–64 as in DeepSeek-style setups), broader temperature sweeps ($T \in \{0.7, 1.0, 1.3, 1.5\}$), and curriculum learning (easy-to-hard schedules)—were not run, so the present evidence does not isolate capacity from exploration effects. Literature suggests curriculum strategies consistently help sub-7B models overcome cold-start barriers [Parashar et al., 2025, Chen et al., 2025b].

5.2 Synthetic vs. Real Data Gap

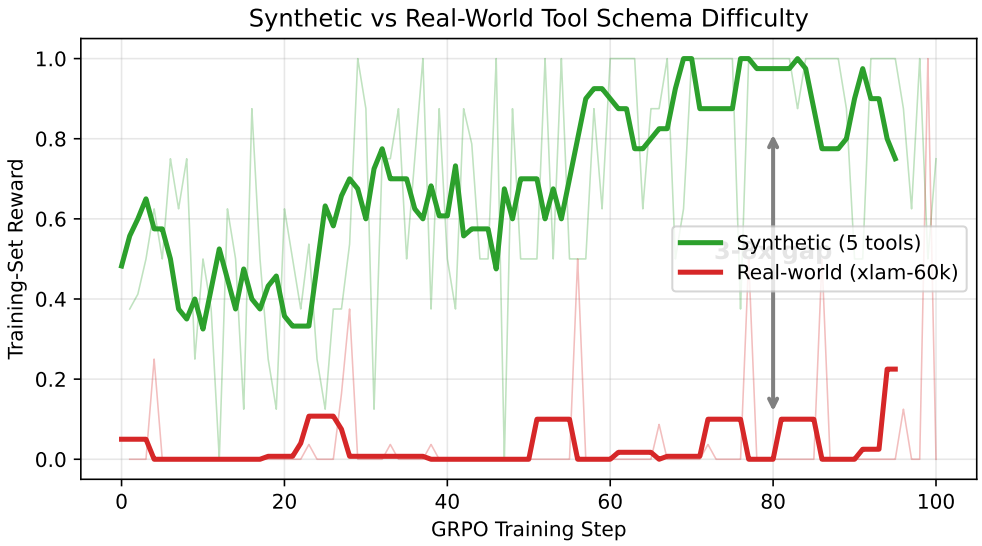

Figure 3: Synthetic vs. real-world tool schema difficulty over 100 GRPO steps (Qwen3-8B). Synthetic 5-tool tasks saturate above 0.8 reward, while real-world xlam-60k schemas remain below 0.2, revealing a 3–8$\times$ difficulty gap.

On tool calling, synthetic 5-tool tasks saturate to reward $>$0.9 within 5 GRPO steps (Figure 3). Salesforce xlam-60k with diverse real-world tool schemas yields rewards of 0.06–0.36 after 100 steps—a 3–8$\times$ difficulty gap. This gap likely reflects schema diversity (5 vs. 60,000 schemas), argument complexity, and distribution shift between curated training data and real-world API specifications.

5.3 Two-Phase Learning Progression

On tool-calling tasks, GRPO training exhibits a characteristic two-phase pattern:

- **Phase 1 (Steps 1–20):** Model learns answer FORMAT compliance (0%$\rightarrow$14% accuracy)
- **Phase 2 (Steps 21–25):** Once format stabilizes, reasoning capability rapidly improves (14%$\rightarrow$58%)

This is analogous to curriculum learning without explicit curricula. However, we note that the two-phase pattern is most pronounced in tool-calling (where format compliance is a distinct, measurable sub-task) and in the arithmetic task (where format validity and correctness are separately tracked). On GSM8K, the pattern is less clear: breakpoints (first step with $>$50% reward) vary from step 1 to step 10 across 5 seeds, and learning trajectories are noisy enough that a clean phase separation is not consistently identifiable. We therefore describe this as a *domain-dependent* observation rather than a universal GRPO property.

5.4 Reward Function Limitations

Our reward functions are deliberately simple and verifiable, but this coarseness introduces known limitations. The tool-calling reward checks JSON validity, tool name, and argument-key presence but does *not* verify argument-value correctness—a model can receive full reward by calling the right function with plausible but wrong values. This gap creates a reward-hacking surface: models may learn to produce syntactically correct but semantically incorrect tool calls. Adding argument-type and value checks, or leveraging outcome reward models such as ToolRM [Wang et al., 2025], would reduce this risk. The math reward is binary (correct/incorrect) with no partial credit for intermediate reasoning steps, which contributes to reward sparsity for weaker models.

5.5 Safety Considerations

GRPO-trained tool-calling models introduce safety risks that warrant explicit discussion. First, **incorrect tool execution**: a model that reliably produces valid JSON may still call the wrong function or supply dangerous arguments (e.g., deleting data instead of reading it). Our reward function does not penalize semantically harmful tool calls. Second, **persistent loops**: although the -0.30 penalty for repeated tool calls eliminated looping in our experiments, more complex multi-step workflows may require explicit loop-detection safeguards. Third, **hallucinated capabilities**: models may generate calls to tools that do not exist in the available schema, particularly when transferring from synthetic to real-world tool sets. Future work should incorporate hallucination and loop penalties validated by standardized evaluators, and evaluate safety under adversarial prompting.

Across all domains, our reporting should be interpreted with the underlying data splits and compute budgets in mind. Tool and GSM8K numbers are primarily computed on training-prompt distributions with fresh rollouts; only the preliminary code-generation result uses a held-out benchmark subset. Typical Tinker runs use batch size 2, group size 4–16, 30–100 optimization steps, and cloud A100 time on the order summarized in the appendix. We do not yet report total tokens processed per run or full GPU-hours per experiment, which is another limitation for strict efficiency comparison.

6 Related Work

GRPO and critic-free RL. GRPO was introduced by Shao et al. [2024] who achieved 88.2% on GSM8K with a 7B model using group size G=64. Several contemporary methods also eliminate the critic: RLOO [Ahmadian et al., 2024] uses a leave-one-out baseline and matches PPO in stability for some RLHF tasks, while REINFORCE++ [Hu et al., 2025] adds KL-shaped, z-scored advantages. Step-wise variants address GRPO's coarse sequence-level credit assignment: S-GRPO [Shen et al., 2025] down-weights imbalanced groups to reduce noise, and StepGRPO [Huang et al., 2025] introduces dense intermediate rewards per reasoning step. We do not yet compare empirically to these alternatives; doing so is a priority for future work.

Preference-based methods. DPO [Rafailov et al., 2024] and Step-DPO [Lai et al., 2024] provide stable offline optimization without online rollouts. Step-DPO demonstrated $\sim$+3% MATH gains for $>$70B models with as few as 10K step-wise preference pairs. Li et al. [2024] showed LLaMA-2-7B reaches 82.6% GSM8K with synthetic SFT alone. These methods establish efficiency baselines that calibrate whether GRPO's online exploration is necessary at small scale.

Tool-use evaluation. Tool-calling evaluation has advanced rapidly: ToolRM [Wang et al., 2025] provides outcome reward models for structured tool-call assessment, and FC-RewardBench [Chen et al., 2025a] offers pairwise discrimination benchmarks for single-turn function calling. For multi-turn evaluation, proxy-state frameworks [Liu et al., 2025] judge tool-chaining quality via final-state inspection with LLM judges, providing hallucination diagnostics absent from our custom rubric. Our tool-calling scores (0.72–0.92) use a hand-authored scenario set without these standardized evaluators; adopting them would strengthen reproducibility.

Curriculum and exploration for small models. Curriculum-learning strategies consistently help sub-7B models overcome cold-start and sparse-reward barriers: easy-to-hard schedules [Parashar et al., 2025], adaptive bandit curricula [Chen et al., 2025b], and progressive length constraints [Hammoud et al., 2025] all yield measurable gains over random training. These approaches are directly relevant to our 3B failure mode (Section 5.1).

Parameter-efficient fine-tuning. Our QLoRA baseline [Dettmers et al., 2023] enables single-GPU training via NF4 quantization. Newer methods target different efficiency frontiers: QR-Adaptor [Zhou et al., 2025] jointly optimizes per-layer bitwidth and LoRA rank (+4.89% GSM8K over prior quantized-LoRA), while LoTA-QAF [Chen et al., 2025c] provides losslessly mergeable ternary adapters with 1.7–2.0× inference speedups. We retain QLoRA as our primary low-friction choice but note that a bounded comparison to these alternatives would sharpen efficiency claims.

Our work extends these findings to smaller models (0.5B–8B) and multiple task domains, providing systematic empirical characterization of GRPO's failure modes and scaling properties. Our current positioning is partial: we study a practical GRPO pipeline but not yet against the full contemporary comparison set outlined above.

7 Planned Experiments and Standardized Evaluation Roadmap

We outline a concrete experimental plan to address the evaluation and ablation gaps identified above, organized by priority.

Phase 1: Held-out evaluation (immediate). (a) Recover trained LoRA adapters from Tinker checkpoints and evaluate on the full GSM8K test set (1,319 examples) with greedy decoding, reporting pass@1 with bootstrapped 95% CIs over 3+ seeds. (b) Evaluate on MATH test set (5,000 examples) for transfer assessment. (c) Relabel all existing figures as "training-set reward" and add paired test-set plots.

Phase 2: Exploration and sparsity ablations. (a) Group size sweep: $G \in \{16, 32, 64, 128\}$ on GSM8K for both 3B and 8B models, measuring zero-advantage fraction and reward density per group size. (b) Temperature sweep: $T \in \{0.7, 1.0, 1.3, 1.5\}$ to probe whether higher sampling temperatures rescue 3B learning. (c) Lightweight KL anchoring: $\beta \in \{0.01, 0.05, 0.1\}$ with logged KL/entropy trajectories. (d) Curriculum: easy-to-hard schedules on GSM8K sorted by solution length.

Phase 3: Standardized benchmarks and baselines. (a) Tool calling: adopt FC-RewardBench-style pairwise discrimination and proxy-state final-answer evaluation with LLM judges; report judge reliability. (b) Code generation: run canonical HumanEval (164 problems) and MBPP harnesses with pass@1/pass@k and 95% CIs. (c) Baselines: add SFT-only, DPO, and (if feasible) RLOO/REINFORCE++ under matched compute budgets.

Phase 4: MoE and PEFT diagnostics. (a) MoE routing telemetry: router entropy, per-expert load coefficient of variation, router-shift ratio. (b) LoRA rank ablation: $r \in \{8, 16, 32, 64\}$ with memory/throughput/accuracy reported side-by-side. (c) Bounded comparison to QR-Adaptor or LoTA-QAF on one model/task pair.

Estimated scope: ∼20–30 runs over 4–6 weeks, \$500–\$1000 in compute. Phases 1–2 can begin immediately from existing checkpoints and logs.

8 Conclusion

We presented an empirical study of GRPO for fine-tuning small language models (0.5B–8B) on tool calling, code generation, and mathematical reasoning. Our key findings, with scope qualifiers:

1. **Tool calling:** GRPO transforms non-tool-calling models into reliable structured callers (0%→92% JSON validity), with a -0.30 penalty eliminating persistent loops. *Scope: custom internal evaluation; standardized benchmarks pending.*

2. **Capacity–exploration interaction:** A sharp performance break exists between 3B and $\geq$8B parameters under our setup. The 3B failure stems from reward sparsity (56% zero-reward steps $\Rightarrow$ zero gradient signal) rather than training instability, but this may be confounded by small group size and fixed temperature. *Scope: not yet disentangled from exploration effects.*

3. **Parameter efficiency:** LoRA rank 32 ($\sim$0.1% trainable params) with QLoRA suffices for near-maximal training-set reward on GSM8K, with rank primarily affecting learning speed rather than asymptotic performance.

4. **Synthetic–real gap:** A 3–8$\times$ difficulty gap exists between synthetic 5-tool schemas and diverse real-world tool schemas (xlam-60k), informing dataset design for tool-calling RL.

5. **Training dynamics:** GRPO exhibits a two-phase progression (format compliance $\rightarrow$ reasoning), rapid entropy collapse ($>$3 orders of magnitude), and MoE routing-induced volatility (2.43$\times$ vs. dense). *Scope: training-set metrics only; held-out generalization unknown.*

Limitations. The most significant gap is evaluation: math and tool-use results are training-set reward metrics, not held-out test accuracy; code generation uses a non-standard 50-problem subset; and tool-calling evaluation relies on a custom rubric without standardized judges. Training runs omit KL regularization and entropy diagnostics, and the MoE analysis lacks routing-level telemetry. The concrete experimental roadmap in Section 7 addresses each of these gaps. In its current form, this work is best read as a practitioner-oriented study of GRPO's training dynamics and failure modes at small scale, not as a generalization claim.

Supplementary Material. Additional experimental details, complete run registry (all 17 Tinker runs), training hyperparameters, failure case analysis, and qualitative examples are provided in the Supplementary Appendix.

Acknowledgments

We thank the PES University MTech DSAI program for compute support and the Tinker team for API access. This work was conducted as a capstone project for the MTech DSAI program.

Reproducibility. Training scripts, evaluation harnesses, and training logs are available in the project repository. Trained LoRA adapters are hosted on Tinker (cloud checkpoints) and partially on HuggingFace Hub. Exact prompt templates, tool schemas, and scenario definitions used for internal tool-calling evaluation are not yet packaged as a standalone replication bundle; releasing these artifacts is an explicit next step. The held-out GSM8K evaluation script (`evaluate_gsm8k_test.py`) is included and validated but awaits checkpoint recovery for execution.

References

Arash Ahmadian et al. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms. 2024.

Hao Chen et al. Fc-rewardbench: A benchmark for function calling reward models. 2025a.

Wei Chen et al. Self-evolving curriculum for reinforcement learning in language models. 2025b.

Xiang Chen et al. Lota-qaf: Lossless ternary adaptation via quantization-aligned factorization. 2025c.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized language models. 2023.

Hadi Hammoud et al. Train long, think short: Progressive length constraints for rl. 2025.

Jian Hu et al. Reinforce++: A simple and efficient approach for aligning large language models. 2025.

Yifan Huang et al. Stepgrpo: Learning to reason with step-wise rewards. 2025.

Xianfeng Lai et al. Step-dpo: Step-wise preference optimization for long-chain reasoning. *arXiv preprint arXiv:2412.01939*, 2024.

Chong Li et al. Common 7b language models already possess strong math capabilities. *arXiv preprint arXiv:2403.04706*, 2024.

Zhen Liu et al. Proxy-state evaluation for multi-turn tool-augmented language models. 2025.

Aneesh Parashar et al. Curriculum reinforcement learning for language model reasoning. 2025.

Rafael Rafailov et al. Direct preference optimization: Your language model is secretly a reward model. 2024.

Zhihong Shao et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Yilong Shen et al. S-grpo: Mitigating think-answer mismatch in group relative policy optimization. 2025.

Yangjun Wang et al. Toolrm: Outcome reward models for tool-augmented language models. 2025.

Yifan Zhou et al. Qr-adaptor: Efficient fine-tuning of quantized language models via joint rank-bitwidth optimization. 2025.